Delta Bot: Complete Guide

What it is, how it works, how it was built, and how to deploy it

---

Version 1.0 · November 2026 · for Delta Exchange India perpetual futures (BTCUSD, ETHUSD, SOLUSD)
Project: github.com/Sumit211995/delta_bot

● **Current mode: PAPER TRADING ONLY. No real orders.**

> **Current mode: PAPER TRADING ONLY.** `DRY_RUN=1` is set in `.env`. The bot sends **no real orders**. It simulates every trade with the same entry, stop loss and target it would use with real money, and records the results. Real trading only starts if *you* change `DRY_RUN=0` **and** set `LIVE_CONFIRM=I_UNDERSTAND_THE_RISK`.

Contents

1. What the bot does, in one page
2. The big picture: parts and how they connect
3. One loop of the bot, step by step
4. The strategy in depth: why it buys or sells
5. Stop loss, target and position size, with a worked example
6. Safety layer: how the bot protects your money
7. Paper trading on 30m, 1h and 4h
8. The dashboard
9. Telegram alerts
10. How we built it: the development story
11. Tools used and why
12. Why Python?
13. Test results: what the data says
14. Daily use: commands
15. Configuration reference (`.env`)
16. Deployment: running it 24/7 on a free server
17. Going live later: checklist (not now)
18. Security rules
19. Troubleshooting and FAQ
20. Glossary

1. What the bot does, in one page

The bot watches the price of **BTC, ETH and SOL** on Delta Exchange India. Every time a **4-hour candle closes** (05:30, 09:30, 13:30, 17:30, 21:30 and 01:30 IST), it checks three fixed rules:

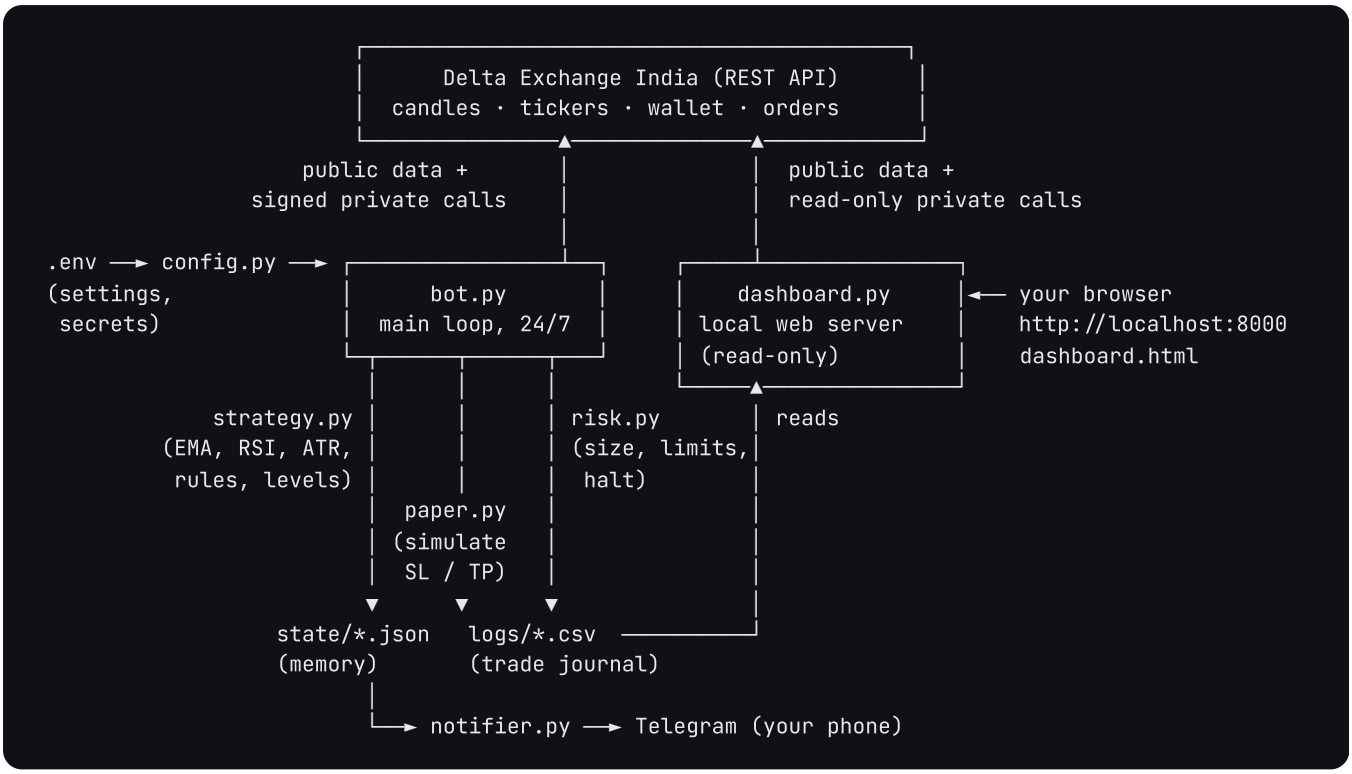
#	Rule	In plain words
1	**Trend**	Is the coin in an uptrend (EMA 50 above EMA 200, price above EMA 200) or a downtrend (the opposite)? In an uptrend it may only **buy**. In a downtrend it may only **sell**.
2	**Pullback**	Did the price dip against the trend? RSI fell **below 40** in an uptrend, or rose **above 60** in a downtrend.
3	**Trigger**	Has the dip ended? RSI just closed **back above 40** (buy) or **back below 60** (sell).

When all three are true, there is a **signal**. The bot then:

- sets a **stop loss** 1.5 × ATR away from the entry (ATR = how much the coin normally moves in one candle),
- sets a **target** 3 × the stop distance away (you risk 1 to make 3),
- picks a **size** so that hitting the stop loses only **0.5% of capital**,
- in **paper mode** (now): records the trade and follows it candle by candle until the stop or target is hit,
- in **live mode** (later): sends **one bracket order** to Delta, with the stop loss and target stored **on the exchange**.

It does **not** predict the future. It follows a fixed rule set that was tested on about 2.5 years of past data and showed a small edge. It wins only about **30%** of trades but makes money because each win is about 3× the size of each loss.

2. The big picture: parts and how they connect



File	Lines	Job
`bot.py`	~355	The main program. Runs forever, checks every 20 seconds whether a candle closed, and places or simulates trades.
`strategy.py`	~140	The "brain": indicators (EMA, RSI, ATR), the buy/sell rules, stop/target levels.
`risk.py`	~80	Position size, daily loss limit, max drawdown halt, trades-per-day cap. Remembers state on disk.
`paper.py`	~25	Simulates the stop/target for paper trades. Same rules as the backtest.
`delta_client.py`	~130	Talks to Delta's API. Signs private requests with your secret key (HMAC-SHA256).
`config.py`	~140	Reads `.env`, applies safe defaults, refuses unsafe settings.
`notifier.py`	~15	Sends Telegram messages.
`dashboard.py` + `dashboard.html`	~440 + ~980	The live web screen (read-only; it can never place an order).
`backtest.py`	~125	Replays history to measure how the strategy would have done.
`train_model.py`	~60	Optional machine-learning filter (off by default; not needed).
`tests/test_core.py`		12 automatic tests for the important logic.
`Dockerfile`, `docker-compose.yml`		Packaging for running on a server.

3. One loop of the bot, step by step

The bot is a simple loop. Every **20 seconds** it does this for each timeframe (4h, then paper 30m and 1h) and each coin:

```
1. Download the last ~400 candles for the coin        (public API, no key needed)
2. Drop the candle that is still forming              (only CLOSED candles are used)
3. Newest closed candle already handled?   ──yes──►  nothing to do, next coin
        │ no
4. Save "handled" to state/ FIRST                     (a crash can never double-trade)
5. Paper: open trade hit its SL or TP?     ──yes──►  close it, update balance, Telegram
6. Compute EMA 50, EMA 200, RSI 14, ATR 14
7. Check the 3 rules on the closed candle  ──no───►  log "no signal", next coin
        │ signal
8. Position already open on this coin?     ──yes──►  ignore the signal
9. Halted? daily loss limit? trade cap?    ──fail──► Telegram "skipped: reason"
10. Entry = mark price; SL/TP from ATR; size from risk %
11. Size < 1 contract?                     ──yes──►  Telegram "skipped: too small"
12. PAPER: record position + log + Telegram
    LIVE : one bracket order (market + SL + TP), then confirm SL/TP exist on Delta
           if not confirmed: close the position, HALT, Telegram alert
```

Why only closed candles? A candle that is still forming can "flash" a signal that disappears before it closes. The backtest only used closed candles, so live trading must too. Otherwise the live results would not match the test. That is why there is no new signal **between** the 4h closes. The dashboard shows a dashed amber **"forming"** hint if a signal *might* appear at the next close, but it only counts once the candle closes.

Why "mark handled" before ordering? If the bot crashed between sending an order and saving, it could restart and send the same order again. Marking first means the worst case is one *missed* trade, never a *duplicate* one. For the same reason, order requests are **never retried automatically** (only read requests are).

4. The strategy in depth: why it buys or sells

The strategy is called **trend pullback**. The idea is old and simple: *trade in the direction of the big trend, but enter after a temporary dip, not at the top of a move.*

4.1 The indicators

Indicator	Formula (simplified)	What it tells us
EMA 50	Exponential moving average of the last ~50 closes	Medium-term direction
EMA 200	Exponential moving average of the last ~200 closes	Long-term direction (the "big trend")
RSI 14	100 − 100 / (1 + average gain / average loss), over 14 candles (Wilder smoothing)	Momentum, 0–100. Low = recently sold off, high = recently pumped
ATR 14	Average of the "true range" (high − low, including gaps) over 14 candles	How much the coin normally moves in one candle, in dollars

On 4h candles, EMA 200 covers about **33 days** and EMA 50 about **8 days**.

4.2 The rules (exact code in `strategy.py → _pullback`)

```
uptrend   = EMA50 > EMA200  and  close > EMA200
downtrend = EMA50 < EMA200  and  close < EMA200

BUY  if uptrend   and previous RSI < 40  and  current RSI ≥ 40     (dip is over, trend resumes up)
SELL if downtrend and previous RSI > 60  and  current RSI ≤ 60     (bounce is over, trend resumes
down)
```

4.3 Why this works (when it works)

- **Trend filter:** markets tend to keep going in the direction of their long-term trend more often than not. Trading against it means fighting the bigger players.
- **Pullback:** buying after a dip gives a better price than buying a breakout, so the stop loss can sit just below normal noise.
- **Trigger on the cross back:** waiting for RSI to turn back avoids "catching a falling knife". The bot enters only once the dip shows signs of ending.
- **3:1 reward:** most trades lose (about 70%), but each winner pays for about three losers. You need a win rate above 25% plus fees to make money; the test showed about 30%.

4.4 Why it does not "predict"

There is no forecast, no AI guess and no news reading. The same candles always give the same answer. That is a strength: you can test it on years of history and know exactly why every trade happened. (An optional ML filter exists in `train_model.py`, but it did not improve results, so it is off.)

5. Stop loss, target and position size, with a worked example

```
stop distance = 1.5 × ATR
BUY : stop loss = entry - stop distance      target = entry + 3 × stop distance
SELL: stop loss = entry + stop distance      target = entry - 3 × stop distance

size (contracts) = floor( capital × risk% ÷ (stop distance × contract value) )
                   also capped so that  size × price × contract value ≤ capital × max leverage
```

Example: ETH sell signal on the 4h chart (paper capital $500)

Step	Value
Entry (mark price)	2,104.00
ATR (4h)	34.70
Stop distance = 1.5 × 34.70	52.05
Stop loss = 2,104 + 52.05	**2,156.05**
Target = 2,104 − 3 × 52.05	**1,947.85**
Contract value (ETHUSD)	0.01 ETH
Loss per contract if stopped = 52.05 × 0.01	$0.52
Risk budget = $500 × 0.5%	$2.50
Size = floor(2.50 ÷ 0.52)	**4 contracts** (0.04 ETH)
Leverage cap = floor(500 × 2 ÷ (2,104 × 0.01))	47 contracts, so it does not limit here
Money at risk if stopped	about **$2.08**
Money made if target hit	about **$6.25**
Fees (0.05% taker + 18% GST, both sides)	about $0.10

Contract sizes on Delta India: BTCUSD = 0.001 BTC, ETHUSD = 0.01 ETH, SOLUSD = 1 SOL. With a very small wallet, some coins cannot be traded at all. For example, one SOL contract risks about $2.40 at a normal stop, which is more than 2% of a $27 wallet. The bot then **skips** the signal instead of over-risking.

6. Safety layer: how the bot protects your money

Protection	Default	What happens
Paper mode by default	`DRY_RUN=1`	No real orders until you change two separate settings.
Live confirmation phrase	`LIVE_CONFIRM`	Live trading refuses to start unless it equals `I_UNDERSTAND_THE_RISK`.
Risk per trade	0.5% (max allowed 2%)	Size is calculated from the stop distance; the config refuses more than 2%.
Capital cap	`BOT_CAPITAL_USD`	The bot sizes from the smaller of the wallet and this cap, even if the wallet is bigger.
Leverage cap	2× (max 10×)	Position value is limited to capital × leverage.
Daily loss limit	−3%	No new trades until the next UTC day.
Max drawdown halt	−10% from peak	The bot **stops permanently** until you review it and reset by hand.
Trades per day	3	Cap per timeframe record.
One position per coin		New signals on a coin are ignored while it has an open position (including your manual trades).
Stop loss on the exchange		Live orders are *bracket* orders: the SL and TP live on Delta's servers, so they work even if your laptop is off.
Protection check		After a live entry, the bot checks that new SL and TP orders exist. If not, it closes the position and halts.
No duplicate orders		Candle marked "handled" before ordering; order requests are never retried.
Fail-safe on errors		If the wallet can't be read, no trade. 20 errors in a row → exit (Docker restarts it).
Panic button	`python bot.py --panic`	Cancels all orders and closes positions on the bot's coins (manual ones too, so be careful).
Read-only dashboard		The web page has no buttons that trade. It only listens on your own machine (127.0.0.1).

7. Paper trading on 30m, 1h and 4h

The strategy was designed and tested for **4h**. On 30m and 1h the backtest **lost money** (none of 91 setting combinations made money after fees). They are still paper-traded side by side so you can **see this for yourself** with live data before trusting any timeframe with real money.

Timeframe	Checked	Mode	Record files
4h ★	every 4 hours	paper now (live later if you choose)	`state/paper_state_4h.json`, `logs/paper_trades_4h.csv`
1h	every hour	**paper only, always**	`state/paper_state_1h.json`, `logs/paper_trades_1h.csv`
30m	every 30 minutes	**paper only, always**	`state/paper_state_30m.json`, `logs/paper_trades_30m.csv`

- Each timeframe starts with its own **$500** paper balance (`BOT_CAPITAL_USD`) and its own limits.
- `PAPER_TIMEFRAMES=30m,1h` in `.env` controls the extra timeframes. Even when 4h goes live, these stay paper-only.
- **How a paper trade closes:** after each closed candle, the bot checks whether the candle's high/low touched the stop or target. If **both** were touched in the same candle, it assumes the **stop** came first (the pessimistic choice, same as the backtest).
- **P&L includes fees:** 0.06% of the trade value on entry and on exit (0.05% Delta taker fee + 18% GST).
- **Entry price** = the mark price at the moment the candle closed, which is close to what a real market order would get.

8. The dashboard

Start it with `python dashboard.py` and open **http://localhost:8000**.

Section	What it shows
Top bar	Mode (PAPER / LIVE), connection status, alerts button 🔕/🔔, light/dark switch
Price strip	Live price of each coin (updates every 2 s, flashes green/red), 24h high/low, funding, volume
Signal cards (one per coin)	**NO SIGNAL** (grey, radar animation, what it is waiting for, countdown) · **▲ BUY SIGNAL / ▼ SELL SIGNAL** (pops in, glows, shows entry / SL / target / size / risk → reward) · **IN TRADE** (live P&L)
Chart	Candles with EMA 50 / EMA 200, **B** (buy) and **S** (sell) markers for every past signal, SL / target / entry lines for open trades, RSI pane with the 40/60 levels. History: 1M, 3M, 6M, 1Y, 2Y, ALL. Timeframes: 30m, 1h, 4h. The last candle moves live.
Signal ticket + rule checklist	The three rules with ✓ / … / ✕, and a ring counting down to the next candle close
Signals across timeframes	A tile per coin per timeframe: NO SIGNAL / BUY / SELL, RSI meter, trend, time to next close
Paper trading · 30m vs 1h vs 4h	A score card per timeframe (balance, P&L, win rate, W/L, average R, max drawdown, open trades) and a **trade journal**: one row per trade with timeframe, coin, side, opened time, **entry, stop loss, target**, status (OPEN / ✓ TARGET / ✕ STOP), exit, closed time, time held, P&L and R. Filters for timeframe / coin / status, and **Download CSV**.
Account & risk	Mode, Delta wallet, risk settings
Strategy history · every past trade (bottom of the page)	The strategy replayed on **all available Delta data** (about 2.5 years) for every coin and every timeframe: a score card per coin × timeframe (trades, win rate, return, profit factor, max drawdown) and a table of **every past trade** with entry, stop loss, target, result, exit, time held, P&L, R and balance. Filters for coin / timeframe / result / side / year, 50 trades per page, **Download CSV**, and **click any row to see that trade on the chart** with its entry/SL/target lines. Simulated with fees and slippage on a separate $500 balance per coin/timeframe.
Open trades / Event log / Delta orders	Open positions with live P&L, the raw bot log, and your real open orders on Delta (read-only)

Other features: `?demo` in the URL shows an example signal (nothing is traded), desktop notifications and a flashing tab title on new signals, phone-friendly layout, all times in IST.

How it gets data: the page asks the local Python server every **2 seconds** for prices and the forming candle (`/api/live`) and every **10 seconds** for signals and paper records (`/api/overview`). The server fetches public data from Delta and caches it briefly so Delta is not flooded. Charts use up to 2 years of candles, fetched in 1,000-candle pieces in parallel and cached for 10 minutes.

9. Telegram alerts

The bot sends a message for: bot started/stopped, every new trade (with timeframe, entry, SL, TP, risk), every stop or target hit (with P&L and new balance), signals that were skipped and why, and errors.

Example: `PAPER 1h SELL 1 SOLUSD @~86.60 SL 88.40 TP 81.20 (risk ~1.80) | downtrend pullback: RSI back below 60`

Setup (already done for you):

1. In Telegram, open **@BotFather** → `/newbot` → choose a name and a username ending in `bot` → copy the **token** into `TELEGRAM_TOKEN=` in `.env`.
2. Open your new bot, tap **Start**, send "hi".
3. Run the chat-id helper from the README (it reads the token from `.env` without printing it) and put the number into `TELEGRAM_CHAT_ID=`.
4. Restart the bot. You should get "Bot started ..." on your phone.

The token only controls the Telegram bot, not your Delta account. Still keep it private.

10. How we built it: the development story

This bot was built step by step in about one working session, using **Claude Code** (an AI coding assistant running in VS Code) together with you. The steps were:

Step	What was done	Why
1. Review	Read the original code: a single-coin EMA-cross bot on 1h	Understand what exists before changing anything
2. Research the API	Studied Delta Exchange India's API: signing, bracket orders, candles, India base URLs, IP whitelist	Orders must be correct the first time. Real money is involved.
3. Honest backtest	Built a fast test engine: entry on the **next** candle's open, stop checked before target, fees + GST + slippage	A backtest that cheats (look-ahead, no fees) gives false hope
4. Found the old strategy loses	EMA-cross on 1h: about −36% (BTC) and −47% (ETH)	Better to know before trading it live
5. Search for a better rule set	Tried 193 combinations (EMA cross, breakouts, pullbacks; different stops and targets) on the **first 60%** of the data only	Tuning on all the data would just "memorise" the past
6. Walk-forward check	Tested the winner on the **last 40%**, which it had never seen	The only honest way to check that an edge is real
7. Picked trend pullback on 4h	Profitable on unseen data for both BTC and ETH	Small but real edge; simple rules
8. Tested 30m and 1h	0 of 91 combinations profitable after fees	Fees eat small-timeframe edges, so these became watch-only
9. Safety layer	Risk manager, halts, protection check, no-duplicate design, panic, live confirmation phrase	A bot must fail safe
10. Multi-coin + paper trading	BTC + ETH (later SOL), paper mode with simulated SL/TP and fees	Practise with no money
11. Dashboard	Read-only web page: live prices, chart, signals, rule checklist; then a professional redesign with animations	See what the bot sees, in real time
12. Real bug found and fixed	Delta's `/v2/orders` **ignores** `product_id` and returns every coin's orders, so BTC showed ETH's orders. Fixed with `product_ids` plus a second check on our side, and a test.	This could have fooled the live "is my stop loss there?" check
13. Added coins carefully	Tested 14 popular coins; only SOL was both profitable and liquid enough	Adding losing coins makes the bot worse

Step	What was done	Why
14. Paper on all timeframes + journal	One bot paper-trades 30m/1h/4h; the dashboard journal shows every trade	Compare timeframes with live data
15. Tests + Git	12 automated tests; every change committed to Git; pushed to GitHub `Sumit211995/delta_bot`	Prove the logic works and keep history

11. Tools used and why

Tool	Used for	Why this one
Python 3	The bot, strategy, backtests, dashboard server, tests	See section 12
pandas	Candle tables, indicators (EMA, RSI, ATR)	The standard tool for time-series data; one line per indicator
NumPy	Fast maths, vectorised signal search over years of data	Hundreds of times faster than plain loops
requests	HTTP calls to Delta and Telegram	Simple, reliable, the most used HTTP library in Python
hmac / hashlib (built in)	Signing private API requests (HMAC-SHA256)	Required by Delta; built into Python, no extra package
http.server (built in)	The dashboard's local web server	No framework needed for a small read-only page; nothing extra to install or secure
HTML + CSS + JavaScript	The dashboard page	Browsers only run JavaScript, so the page itself has to use it. No build step and no framework, just one file.
TradingView Lightweight Charts (open source)	The candle chart with markers and price lines	Fast with years of candles, looks professional, free
Inter / JetBrains Mono (Google Fonts)	Text and numbers on the dashboard	Clean, easy-to-read numbers
pytest	Automatic tests	Python's standard test runner
Docker + Docker Compose	Running on a server	Same environment everywhere; automatic restart; one command to start
Git + GitHub	Version history, backup, deployment source	Every change is recorded and reversible; the server pulls code from GitHub
Telegram Bot API	Phone alerts	Free, instant, no app to build
Delta Exchange India API v2	Market data and orders	The exchange you trade on; public data needs no key
VS Code + Claude Code	Editing; AI pair-programming (code, research, tests, review, screenshots)	Faster development with a second pair of eyes on safety
Headless Google Chrome	Screenshots to check the UI; turning this guide into a PDF	Already on the Mac; renders exactly like the browser
Oracle Cloud Always Free (planned)	24/7 hosting	Free forever VM in India with a fixed public IP

Not used, on purpose: no paid data feeds, no paid libraries, no database server (JSON and CSV files are enough and easy to inspect), and no heavy web framework.

12. Why Python?

The bot itself, the strategy, the tests and the dashboard server are all **Python**. Only the dashboard page uses HTML/JavaScript, because that is the only language a web browser runs. The reasons:

1. **The backtest and the live bot use the same code.** `strategy.py` is imported by both `backtest.py` and `bot.py`. The rules that were tested are exactly the rules that trade. In a two-language setup (e.g. research in Python, trading in Java) the two versions slowly drift apart, and the live results stop matching the test.
2. **The best tools for market data.** pandas and NumPy make indicators one line each and test years of data in seconds. Almost every trading tutorial, exchange example and quant library is in Python.
3. **Speed is not the problem here.** The bot acts once per candle (every 4 hours on the main timeframe) and spends most of its time waiting. Python is more than fast enough. C++/Rust/Go only matter for high-frequency trading in milliseconds, which this is not.
4. **Easy to read and change.** The code reads almost like English, so you can open `strategy.py` and see the exact rules.
5. **Few moving parts.** Three main packages (requests, pandas, numpy). Fewer dependencies mean fewer things to break or attack.
6. **Easy to deploy.** One Docker image based on the official `python:3.11-slim` runs on any Linux server, including the free Oracle VM.

Alternatives and why not: Node.js (good for web, weaker for data analysis), Java/C# (more code for the same thing), Go/Rust (fast, but the speed is not needed and the data tools are weaker), no-code platforms like Tradetron/AlgoTest (monthly fees, less control, and you would need to whitelist their servers on your API key).

13. Test results: what the data says

All tests include fees (0.06% per side), slippage (0.02%) and entry on the **next** candle. Numbers are for the **4h trend-pullback** strategy, 1% risk per trade, Delta India data from late 2023 / early 2024 to November 2026.

Main coins (README backtest, settings tuned on the first 60%, checked on the last 40%):

Coin	Trades	Win rate	Profit factor	Return	Max drawdown
BTCUSD	99	30%	1.17	+14.0%	9.2%
ETHUSD	88	31%	1.23	+15.8%	11.0%

14 popular coins (same settings, never tuned on them):

Coin	Full period PF	Last 40% PF	Decision
ETH	1.30	1.30	✅ traded
SOL	1.13	1.48	✅ traded (added)
BTC	1.18	1.06	✅ traded (weaker lately)
DOGE, BCH	1.29 / 1.54	positive	⚠ not added: only ~$2M daily volume on Delta (bad fills)
BNB, AVAX, TRX	about 1.0	mixed	❌ no edge
XRP, ADA, LINK, DOT, SUI, LTC	below 1.0		❌ lose money

Timeframes: 30m and 1h: **0 of 91** setting combinations were profitable on unseen data after fees. 4h: robust.

How to read this: profit factor (PF) = money won ÷ money lost; above 1.0 is profitable. **1.1–1.3 is a small edge.** Expect losing months, and runs of 8–10 losses in a row. About **6–8 trades per month** across BTC + ETH + SOL on 4h. Past results do not guarantee future results.

14. Daily use: commands

Run these in the project folder (`cd ~/Documents/Project/delta_bot`):

Command	What it does
`caffeinate -i python bot.py`	Start the bot (4h + paper 30m + paper 1h). `caffeinate` stops the Mac sleeping. Stop with Ctrl+C.
`python dashboard.py`	Start the dashboard → open http://localhost:8000
`python bot.py --status`	Paper balance, open trades, wins/losses for every timeframe
`python bot.py --check`	Test the connection: product info, wallet balance, positions, open orders. Places no orders.
`python bot.py --once`	Process the latest closed candle once and exit
`python bot.py --paper-tf none`	Run only the main timeframe (no extra paper timeframes)
`python bot.py --symbols ETHUSD`	Override the coins for this run
`python backtest.py --symbol ETHUSD --resolution 4h`	Re-run the backtest with a per-year breakdown
`python -m pytest -q tests`	Run the automatic tests (should say "12 passed")
`python bot.py --panic`	**Emergency:** cancel all orders and close positions on the bot's coins (manual trades too)

15. Configuration reference (`.env`)

`.env` holds your settings **and secrets**. It is listed in `.gitignore` and must **never** be committed or shared. `.env.example` is the safe template.

Setting	Current / default	Meaning
`DELTA_ENV`	`live`	`live` = api.india.delta.exchange · `testnet` = fake-money test exchange
`DRY_RUN`	`1`	**1 = paper trading (no orders).** 0 = real orders
`LIVE_CONFIRM`	empty	Must be `I_UNDERSTAND_THE_RISK` for real orders on `live`
`DELTA_API_KEY` / `DELTA_API_SECRET`	set	Your Delta API key (Read Data + Trading; **never** Withdrawal)
`SYMBOLS`	`BTCUSD,ETHUSD,SOLUSD`	Coins to trade, comma separated
`RESOLUTION`	`4h`	Main timeframe (the tested one)
`PAPER_TIMEFRAMES`	`30m,1h`	Extra timeframes, paper only
`STRATEGY`	`pullback`	`pullback` (default) or `ema_cross` (old, loses money)
`RSI_PULLBACK`	`40`	Buy level (sell level = 100 − this = 60)
`ATR_STOP_MULT`	`1.5`	Stop distance in ATRs
`REWARD_RISK`	`3.0`	Target = this × stop distance
`BOT_CAPITAL_USD`	`500`	Capital the bot sizes from (and paper starting balance)
`RISK_PER_TRADE`	`0.005`	0.5% risked per trade (max 0.02)
`MAX_LEVERAGE`	`2`	Leverage cap (1–10)
`DAILY_LOSS_LIMIT`	`0.03`	Stop new trades for the day at −3%
`MAX_DRAWDOWN`	`0.10`	Halt permanently at −10% from peak
`MAX_TRADES_PER_DAY`	`3`	Per timeframe record
`FEE_RATE`	`0.0006`	Per side, used for paper P&L and backtests
`TELEGRAM_TOKEN` / `TELEGRAM_CHAT_ID`	set	Phone alerts

16. Deployment: running it 24/7 on a free server

Right now the bot runs on your laptop, so it only works while the laptop is on, awake and online. A server runs it 24/7. **Deploy in paper mode first (`DRY_RUN=1`)** and keep it that way until you decide to go live.

16.1 What the server needs

- **Always on**, with internet
- A **fixed public IP**: Delta API keys only work from whitelisted IPs, and since April 2026 Indian brokers also require a static IP for API trading
- **Persistent disk** for `state/` and `logs/` (the bot's memory and trade journal)
- About **150 MB RAM** for both containers. Any small VM is enough.

16.2 Options

Option	Cost	Fixed IP	Verdict
Oracle Cloud Always Free (Mumbai / Hyderabad)	Free	Yes	**Recommended**
Your laptop	Free	No (mobile hotspot IP changes)	Fine for paper testing only
Paid VPS (AWS Lightsail Mumbai, DigitalOcean Bangalore, etc.)	about $4–6 / month	Yes	Good if Oracle signup fails
Railway / Render / Heroku-style platforms	about $5+ / month	Usually not	Not suitable (no fixed outbound IP on cheap plans)

16.3 Before you start (on your laptop)

1. Push the latest code to GitHub: `git push -u origin main` (use your Sumit211995 personal access token as the password).
2. Make sure `.env` is **not** on GitHub (it is in `.gitignore`). You will copy it to the server separately.
3. Keep `DRY_RUN=1` in the `.env` you copy.

16.4 Step by step: Oracle Cloud Always Free

Step 1: Create the account

1. Go to **cloud.oracle.com** → *Sign up*. A card is needed for identity verification; Always Free resources are not charged.
2. Choose home region **India West (Mumbai)** or **India South (Hyderabad)**. It cannot be changed later.

Step 2: Create the virtual machine

1. Menu → *Compute* → *Instances* → **Create instance**.
2. Image: **Canonical Ubuntu 22.04** (or 24.04).
3. Shape: **VM.Standard.E2.1.Micro** (AMD, 1 GB RAM) or **VM.Standard.A1.Flex** (Ampere ARM, 1 OCPU / 6 GB). Both are marked "Always Free eligible". If Ampere says "out of capacity", use E2.1.Micro or try again later.

4. Networking: keep the default VCN, **assign a public IPv4 address**.
5. SSH keys: **Generate a key pair** and **download the private key** (e.g. `oracle.key`). Keep it safe.
6. Click **Create** and wait until the instance is *Running*. Copy the **Public IP address**.
7. Optional but recommended: in the instance's VNIC → IPv4 addresses, change the public IP to a **reserved** IP so it never changes, even if the instance is recreated.

Step 3: Whitelist the server IP on Delta

1. Delta Exchange India → *Account* → **API Keys** → edit your key (or create a new one just for the server).
2. Add the server's public IP to the **whitelisted IPs**. Permissions: **Read Data + Trading** only. **Never enable Withdrawal.**

Step 4: Connect to the server (from your Mac terminal)

```
chmod 400 ~/Downloads/oracle.key
ssh -i ~/Downloads/oracle.key ubuntu@<SERVER_IP>
```

Step 5: Install Docker, Git and time sync

```
sudo apt update && sudo apt -y upgrade
sudo apt install -y docker.io docker-compose-v2 git chrony
sudo systemctl enable --now docker chrony
sudo usermod -aG docker ubuntu
exit                                  # log out and in again so the docker group applies
```

Time sync (`chrony`) matters: Delta rejects signed requests if the server clock is more than a few seconds off.

Step 6: Get the code

```
ssh -i ~/Downloads/oracle.key ubuntu@<SERVER_IP>
git clone https://github.com/Sumit211995/delta_bot.git
cd delta_bot
mkdir -p state logs
```

(If the repo is private, use `https://<your-token>@github.com/Sumit211995/delta_bot.git` once, or set up an SSH deploy key.)

Step 7: Copy your `.env` safely (run this on your **Mac**, not on the server)

```
scp -i ~/Downloads/oracle.key ~/Documents/Project/delta_bot/.env ubuntu@<SERVER_IP>:~/delta_bot/.env
```

Then on the server: `chmod 600 ~/delta_bot/.env` and check `grep DRY_RUN ~/delta_bot/.env` shows `DRY_RUN=1`.

Step 8: Start the bot and dashboard

```
cd ~/delta_bot
docker compose up -d --build          # builds the image, starts both containers, restarts them
on reboot/crash
docker compose ps                     # both should be "running"
docker compose logs -f delta-bot      # watch the bot (Ctrl+C only stops watching)
```

You should get **"Bot started ... PAPER 4h ... also paper-trading 30m,1h"** on Telegram.

Step 9: Check the connection from the server

```
docker compose exec delta-bot python bot.py --check      # wallet balance + positions; no orders
docker compose exec delta-bot python bot.py --status     # paper records
```

If `--check` says "IP not whitelisted" or "unauthorized", go back to Step 3.

Step 10: Open the dashboard from your laptop

The dashboard listens only on the server itself (`127.0.0.1:8000`), so nobody on the internet can see it. Open a secure tunnel from your Mac:

```
ssh -i ~/Downloads/oracle.key -N -L 8000:localhost:8000 ubuntu@<SERVER_IP>
```

Leave that terminal open and browse to **http://localhost:8000**. Do **not** open port 8000 in Oracle's firewall.

Step 11: Stop the laptop copy

Only **one** bot should run per Delta account. Once the server bot works, stop the one on your laptop (Ctrl+C). You can still run `python dashboard.py` locally to view prices; it reads your laptop's paper records, not the server's.

16.5 Running and maintaining the server

Task	Command (on the server, in `~/delta_bot`)
See if it is running	`docker compose ps`
Watch logs	`docker compose logs -f --tail 100 delta-bot`
Paper results	`docker compose exec delta-bot python bot.py --status`
Update to the latest code	`git pull && docker compose up -d --build`
Restart	`docker compose restart`
Stop everything	`docker compose down` (live positions keep their exchange SL/TP)
Emergency close (live only)	`docker compose exec delta-bot python bot.py --panic`
Back up records to your Mac	`scp -i ~/Downloads/oracle.key -r ubuntu@<SERVER_IP>:~/delta_bot/{state,logs} ~/delta_backup/`
Change a setting	`nano .env` then `docker compose up -d`

- **Auto-restart:** `restart: unless-stopped` in `docker-compose.yml` restarts the bot after a crash or a server reboot.
- **Memory:** `state/` and `logs/` are mounted from the server disk, so updates and restarts keep all records.

- **Monitoring:** Telegram tells you about starts, stops, trades and repeated errors. If you get no "Bot started" message after an update, check the logs.
- **Security:** keep Ubuntu updated (`sudo apt update && sudo apt -y upgrade` monthly), never open port 8000, only use SSH keys.

16.6 Deploy checklist

- ☐ Code pushed to GitHub, `.env` **not** in the repo
- ☐ Oracle VM running in Mumbai/Hyderabad, public IP noted (reserved if possible)
- ☐ Server IP whitelisted on the Delta API key (Read + Trade, **no Withdrawal**)
- ☐ Docker, Git and chrony installed
- ☐ `.env` copied with `scp` , `chmod 600` , `DRY_RUN=1`
- ☐ `docker compose up -d --build` → both containers running
- ☐ `bot.py --check` works from the server
- ☐ Telegram "Bot started" received
- ☐ Dashboard opens through the SSH tunnel
- ☐ Laptop bot stopped (only one bot running)

17. Going live later: checklist (not now)

Do not do this yet. First collect at least 2–4 weeks of paper results. When you decide to go live:

1. Paper results on 4h look in line with the backtest (win rate about 25–35%, losses kept small).
2. Use a **separate Delta sub-account** for the bot, or close manual trades on BTC/ETH/SOL. The bot ignores coins with an open position, and `--panic` closes manual positions too.
3. Fund it with an amount you can afford to lose. With about $27, only ETH fits (`SYMBOLS=ETHUSD` , `BOT_CAPITAL_USD=27` , `RISK_PER_TRADE=0.02`). About ₹15–20k gives the bot room to trade BTC, ETH and SOL at normal risk.
4. Optionally test the real order flow once on **testnet** (`DELTA_ENV=testnet` , testnet keys).
5. Set `DRY_RUN=0` and `LIVE_CONFIRM=I_UNDERSTAND_THE_RISK` , then restart. 30m and 1h stay paper-only automatically.
6. Watch the first live trade: check on Delta that the stop loss and take-profit orders appear.

18. Security rules

- **Never share** the Delta API secret, the Telegram token or the GitHub token. Never paste them in chats or screenshots.
- **Never enable Withdrawal** on the Delta API key. Without it, even a stolen key cannot take money out.
- **Whitelist only your own IPs** (laptop/server). Do not add third-party platform IPs you do not use.
- `.env` stays out of Git (`.gitignore`) and has `chmod 600` on the server.
- The dashboard is read-only and bound to `127.0.0.1` . Reach it on a server only through an SSH tunnel.
- If a secret leaks: delete that API key in Delta immediately, create a new one, update `.env` , restart.

19. Troubleshooting and FAQ

Problem / question	Answer
No trades for days	Normal. 4h gives about 6–8 trades a month across 3 coins; some weeks have none. 30m/1h trade more often.
"Signal skipped: size < 1 contract"	The capital is too small for that coin's stop distance. Expected for SOL with a small wallet.
`--check` says unauthorized / IP not whitelisted	Your current IP (or the server's) is not on the API key's whitelist. Mobile hotspot IPs change often.
"Signature expired"	The computer's clock is off. Turn on automatic time (Mac) or install chrony (server).
Bot says HALTED	Max drawdown was hit, or the protection check failed. Read the reason in `--status`, review, then delete the matching `state/*.json` file to reset.
The dashboard shows nothing	Is `python dashboard.py` running? Refresh the page. Check that internet works.
Will the stop loss work if my laptop is off?	**Live:** yes, SL/TP are stored on Delta. **Paper:** paper trades are only checked while the bot runs; it catches up on the candles it missed when restarted.
Can the bot trade Indian stocks (Upstox)?	Not yet. It needs a separate broker adapter, a backtest on Indian markets, and a static IP under SEBI's 2026 rules.
Is the API free?	Yes. Delta and Telegram APIs are free. You pay only normal trading fees (0.05% taker + 18% GST) on real trades.
Can I add more coins?	Test them first (`backtest.py --symbol X`). Of 14 popular coins tested, only BTC, ETH and SOL qualified.

20. Glossary

Term	Meaning
Candle	Open, high, low and close price for one time period (e.g. 4 hours)
Perpetual future	A contract that follows the coin price with no expiry date; you can buy (long) or sell (short)
Contract value	How much coin one contract represents (ETHUSD: 0.01 ETH)
Mark price	The fair price Delta uses for P&L and stop triggers (resists sudden spikes)
EMA	Exponential moving average: an average that weights recent prices more
RSI	Relative Strength Index, 0–100 momentum gauge
ATR	Average True Range: typical movement per candle
Bracket order	An entry order with a stop loss and take profit attached, stored on the exchange
R / R-multiple	Result divided by the amount risked: +3R = won three times the risk; −1R = full stop loss
Profit factor	Total money won ÷ total money lost
Drawdown	The fall from the highest balance to a later low
Backtest	Replaying past data to see how rules would have performed
Walk-forward test	Tune on older data, check on newer unseen data
Paper trading	Simulated trading with live prices and no real money
Slippage	The difference between the expected price and the price you actually get
HMAC-SHA256	The signing method that proves an API request came from your key

This guide describes software for personal use. It is not financial advice. Trading leveraged crypto futures can lose money quickly; only trade what you can afford to lose.